

CINESTREAM

Arquitectura del Repositorio Genérico de Datos y Matriz de Tipado

Tarea 4 — Patrones de diseño y tipado seguro

Estudiante: Carlos Daniel Figueroa Salazar

Carné: 0904-23-8889

Sección: A

Universidad: Universidad Mariano Gálvez de Guatemala

Docente: Kelvin Castillo

Asignatura: Desarrollo Web

Proyecto: Plataforma de visualización multimedia CineStream

Fecha: 7 de agosto de 2026

Repositorio:

<https://github.com/Hyground/Laboratorio1>

1. Resumen ejecutivo

Se refactorizó CineStream para sustituir la administración directa de arreglos por un repositorio genérico en memoria. La solución conserva el tipo concreto de cada entidad, separa los contratos externos de los modelos internos y sanea payloads incompletos antes de incorporarlos al catálogo.

Resultado: una misma implementación administra `Movie`, `Series` y `Documentary`, sin `any`, sin supresiones y con compilación TypeScript estricta libre de errores.

Objetivos alcanzados

- Repositorio `DataCatalogManager<T extends { id: string | number }>`.
- Retornos polimórficos que preservan `T`, `T[]` y `T | undefined`.
- Uso funcional de `Partial`, `Pick` y `Omit`.
- Frontera de red basada en `unknown` y validación propiedad por propiedad.
- Saneamiento mediante valores predeterminados de negocio.
- Integración real del repositorio en los filtros, búsquedas y renderizado.

2. Arquitectura implementada

La entidad base `CatalogEntity` concentra el contrato compartido. Las interfaces especializadas agregan información de dominio sin obligar al repositorio a conocerla.

```
export interface CatalogEntity {
  id: string | number;
  title: string;
  synopsis: string;
  posterUrl: string;
  releaseYear: string;
  rating: number;
}

export interface Movie extends CatalogEntity { /* campos de película */ }
export interface Series extends CatalogEntity { /* temporadas y episodios */ }
export interface Documentary extends CatalogEntity { /* tema y duración */ }
```

Responsabilidades por capa

Capa	Responsabilidad	Archivos principales
DTO	Representar el contrato externo y admitir campos parciales tras la inspección.	dtos/movie-db.dto.ts
Mapper	Sanear y convertir el DTO en una entidad completa del dominio.	mappers/movie.mapper.ts
Entidad	Definir datos confiables consumidos internamente.	entities/*.entity.ts
Repositorio	Encapsular almacenamiento, consultas y mutaciones tipadas.	repository/data-catalog.manager.ts
Servicio	Orquestar APIs, normalizar payloads y enriquecer metadatos.	service/movie.service.ts
Interfaz	Filtrar y renderizar entidades ya saneadas.	app.ts

3. Generic Repository

La restricción genérica garantiza en compilación que toda entidad posea una identidad compatible. Internamente se utiliza `Map<T['id'], T>` para obtener acceso directo por identificador y evitar duplicados.

```
export class DataCatalogManager<T extends { id: string | number }> {
  private readonly items = new Map<T['id'], T>();

  public getAll(): T[];
  public findById(id: T['id']): T | undefined;
  public findBy<K extends keyof T>(field: K, value: T[K]): T[];
  public filter(predicate: (item: Readonly<T>) => boolean): T[];
  public update(id: T['id'], changes: CatalogUpdate<T>): T | undefined;
  public remove(id: T['id']): boolean;
}
```

Preservación matemática del tipo

Instancia	Consulta	Retorno inferido
DataCatalogManager<Movie>	filter(...)	Movie[]
DataCatalogManager<Series>	findById(...)	Series undefined
DataCatalogManager<Documentary>	findBy('educational', true)	Documentary[]

El par genérico `K extends keyof T / T[K]` impide buscar con una propiedad inexistente o comparar una propiedad con un valor incompatible. Por ejemplo, `findBy('rating', 'alta')` es rechazado porque `rating` requiere un número.

4. Matriz de Utility Types

Utilidad	Definición aplicada	Caso de uso	Riesgo evitado
Pick	Pick<T, 'id'>	CatalogIdentity<T> y selección de valores predeterminados del mapper.	Solicitar una identidad con campos ajenos o exponer datos innecesarios.
Omit	Omit<T, 'id'>	CatalogCreate<T> recibe los datos mientras el ID se proporciona por separado.	Duplicar o contradecir la identidad al crear una entidad.
Partial + Omit	Partial<Omit<T, 'id'>>	CatalogUpdate<T> permite parches de uno o varios campos.	Exigir objetos completos en una actualización o permitir modificar el ID.
Partial	Partial<MovieDbMovieDto>	IncompleteMovieDbDto representa datos de red inspeccionados pero incompletos.	Tratar como confiable un contrato externo corrupto.

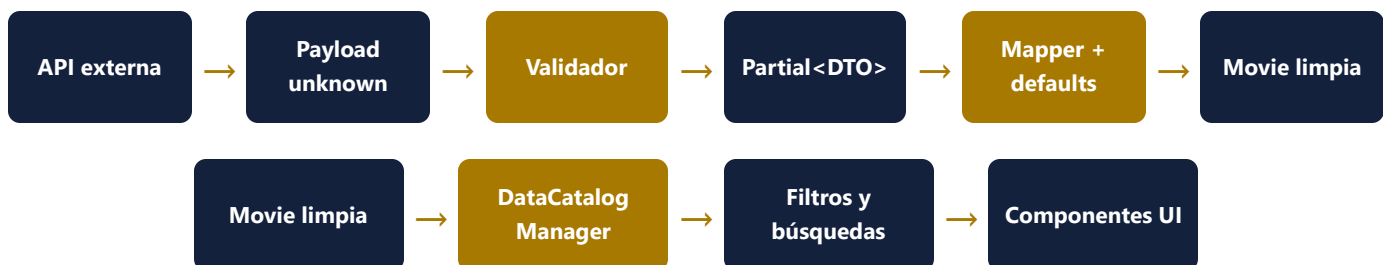
Decisión de seguridad: la mutación reconstruye la entidad con `id: current.id`. Incluso si un consumidor intenta introducir el identificador mediante una conversión externa, el repositorio preserva la identidad original durante la ejecución.

5. Interface frente a type

Criterio	interface	type	Elección en CineStream
Modelado de objetos	Expresa contratos nominalmente legibles y extensibles.	También modela objetos, pero destaca en composiciones.	interface para DTOs y entidades.
Herencia	Utiliza <code>extends</code> de manera directa.	Utiliza intersecciones con <code>&</code> .	interface para Movie, Series y Documentary sobre CatalogEntity.
Utility Types	No representa directamente el resultado calculado de una utilidad.	Puede asignar tipos derivados y combinaciones.	type para CatalogCreate, CatalogUpdate e IncompleteMovieDbDto.
Uniones	No modela uniones por sí sola.	Ideal para uniones literales.	type para Language y Genre.
Fusión de declaraciones	Permite declaration merging.	No fusiona declaraciones.	Las entidades podrían ampliarse por integración; los tipos calculados deben permanecer cerrados.
Intención semántica	"Este objeto debe cumplir este contrato".	"Este nombre representa este cálculo o conjunto de tipos".	La selección comunica el papel arquitectónico de cada construcción.

La decisión no se basa en preferencia estilística. Las entidades y respuestas son contratos estructurales abiertos a extensión; las operaciones de creación y actualización son transformaciones calculadas a partir de dichos contratos.

6. Flujo seguro de datos



Tratamiento del caos de red

1. `fetchJson<unknown>` evita afirmar prematuramente que la respuesta cumple el DTO.
2. `isRecord` descarta primitivos y valores nulos.
3. `toIncompleteMovieDto` copia exclusivamente propiedades cuyo tipo fue comprobado.

4. El mapper normaliza textos vacíos, identificadores ausentes, años inválidos y puntuaciones no numéricas.
5. La entidad completa resultante es la única forma admitida por el repositorio y la interfaz.

7. Valores predeterminados de negocio

Campo problemático	Validación	Valor seguro
id ausente o no finito	<code>typeof id === 'number' && Number.isFinite(id)</code>	Índice estable dentro del género.
title ausente o vacío	<code>trim()</code> con fallback	"Sin título".
synopsis ausente	Enriquecimiento externo o fallback	"Sinopsis no disponible".
posterURL corrupta	Solo se acepta string no vacío	Portada visual generada por CSS.
year ausente o no finito	Comprobación numérica	"Año no disponible".
rating inválido	<code>Number.isFinite</code> y valor no negativo	0 internamente; "Sin puntuación" en UI.

8. Configuración estricta

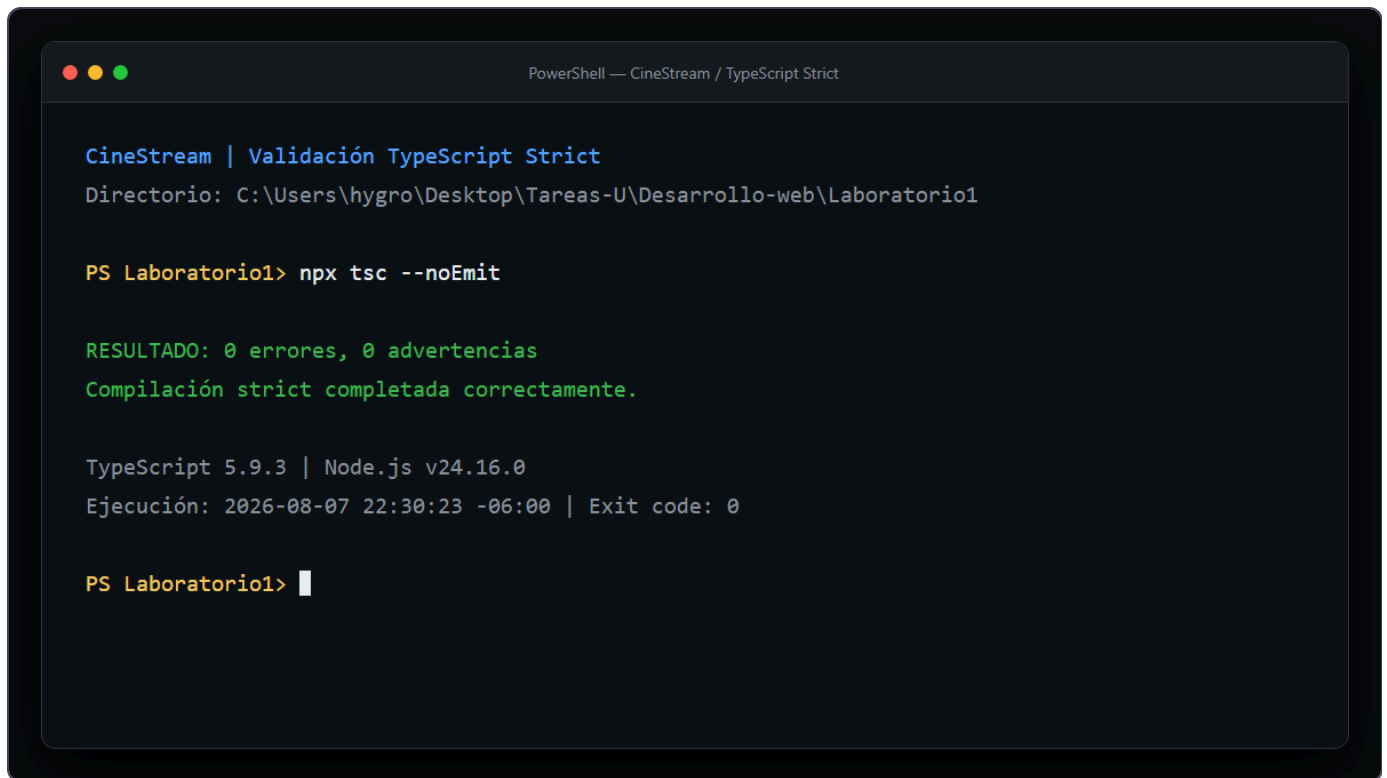
La configuración activa controles superiores al requisito base:

```
"strict": true,  
"noImplicitAny": true,  
"strictNullChecks": true,  
"noUncheckedIndexedAccess": true,  
"exactOptionalPropertyTypes": true,  
"noFallthroughCasesInSwitch": true,  
"noEmit": true,  
"noEmitOnError": true
```

`noUncheckedIndexedAccess` obligó a verificar resultados obtenidos por índice, mientras que `exactOptionalPropertyTypes` evitó asignar explícitamente `undefined` a propiedades opcionales. Estas reglas detectaron riesgos reales durante la refactorización.

9. Evidencia de compilación

Se ejecutó `npx tsc --noEmit` sobre la base completa, incluyendo la demostración polimórfica ubicada en `tests/`.

A terminal window with a dark background and light-colored text. The title bar at the top reads "PowerShell — CineStream / TypeScript Strict". The terminal content shows a directory path, a command to run TypeScript with strict mode and no emit, the successful result of the compilation, and the exit code.

```
CineStream | Validación TypeScript Strict
Directorio: C:\Users\hygro\Desktop\Tareas-U\Desarrollo-web\Laboratorio1

PS Laboratorio1> npx tsc --noEmit

RESULTADO: 0 errores, 0 advertencias
Compilación strict completada correctamente.

TypeScript 5.9.3 | Node.js v24.16.0
Ejecución: 2026-08-07 22:30:23 -06:00 | Exit code: 0

PS Laboratorio1> 
```

Figura 1. Ejecución verificada: TypeScript 5.9.3, código de salida 0, cero errores.

10. Verificación de la rúbrica

Criterio	Evidencia implementada	Estado
Abstracción genérica y DTOs	Repositorio restringido por identidad, tres entidades polimórficas, DTO aislado y mapper seguro.	Completo
Utility Types	Partial, Pick y Omit aplicados a actualización, identidad, creación y payload incompleto.	Completo
Reporte PDF	Análisis interface/type, flujo conceptual, matriz y evidencia estricta.	Completo
Prohibición de any	Búsqueda global sin coincidencias; frontera externa tratada como unknown.	Completo
Compilación	npx tsc --noEmit, exit code 0.	Completo

11. Conclusiones

El patrón Generic Repository reduce duplicación sin sacrificar información de tipos: cada instancia conserva las propiedades particulares de su entidad. La frontera basada en `unknown`, el DTO parcial saneado y el mapper completo impiden que datos defectuosos se propaguen hacia la interfaz.

Las Utility Types funcionan como reglas de negocio compilables. `CatalogUpdate<T>` expresa que una modificación es parcial y no puede alterar la identidad; `CatalogCreate<T>` separa la creación de la asignación del ID; y `CatalogIdentity<T>` limita las consultas de identidad al dato indispensable.

Conclusión final: CineStream queda preparado para incorporar nuevas familias de contenido sin duplicar infraestructura y con una defensa estática y dinámica frente a cambios en contratos externos.

Repositorio de entrega

<https://github.com/Hyground/Laboratorio1>

Documento generado como parte de la Tarea 4 de Desarrollo Web. El código fuente, la evidencia y la versión HTML de este reporte se incluyen en el mismo repositorio.